SOFTWARE ARCHITECTURE

My Site — Architecture

arc42 / MADR architecture documentation

2026-06-22

Table of Contents

Executive Summary

This document describes the software architecture of the **My Site** platform. It follows an arc42 inspired structure and records the main architectural decisions using the MADR template.

The platform is a static documentation portal generated with Docusaurus, served behind a reverse proxy and continuously delivered through a CI/CD pipeline. Its primary quality goals are **availability**, **maintainability** and **security**.

Aspect	Summary
Purpose	Public documentation and architecture portal
Architecture style	Static site generation (Jamstack)
Delivery	Containerised build, deployed via GitHub Actions
Key qualities	Availability, maintainability, security

Stakeholders

The following stakeholders have an interest in the architecture of the system.

Role	Concern	Expectation
Product Owner	Business value and roadmap	Predictable delivery of features
Developers	Maintainability	Clear structure and documentation
Operations	Availability and observability	Stable, monitorable deployments
Security Officer	Confidentiality and integrity	Compliance with security baselines
End Users	Usability and performance	Fast, accessible documentation

Stakeholders are consulted whenever a decision affects their concerns, and every significant decision is captured as an Architectural Decision Record.

The overall system landscape

The system is composed of a small number of cooperating building blocks. The documentation content is authored in Markdown, compiled into static assets, and served to users through a content delivery layer.

The main elements are:

- **Authoring** — Markdown and MDX sources kept under version control.
- **Build** — Docusaurus compiles the sources into static HTML, CSS and JS.
- **Delivery** — a reverse proxy (Caddy) serves the static assets.
- **Automation** — a CI/CD pipeline builds, tests and publishes every change.

External dependencies are intentionally minimal to keep the runtime simple and to maximise availability.

Architectural Decision Records

This section records the significant architectural decisions using the MADR format. Each decision lists its context, the options considered and the outcome.

Hosting Decision

Context and Problem Statement

We need a hosting approach that is cheap to operate, highly available and simple to reproduce across environments. How should the documentation portal be hosted?

Considered Options

- Static hosting behind a reverse proxy (Caddy)
- Managed platform-as-a-service
- Self-managed virtual machine

Decision Outcome

Chosen option: **static hosting behind a reverse proxy**, because it minimises the runtime attack surface, is inexpensive and integrates cleanly with the CI/CD pipeline.

Localization Requirements for Software System

Context and Problem Statement

The documentation must be available in several languages without duplicating the build infrastructure.

Considered Options

- Built-in Docusaurus i18n
- External translation management system
- Manual per-language branches

Decision Outcome

Chosen option: **built-in Docusaurus i18n**, as it keeps translations close to the source and is supported out of the box.

Availability Level for Application

Context and Problem Statement

What availability target should the application meet, and how do we achieve it?

Considered Options

- Best effort (no formal target)
- 99.9% availability
- 99.99% availability with multi-region delivery

Decision Outcome

Chosen option: **99.9% availability**, achieved through static delivery and a content delivery network, which balances cost against user expectations.

Non Functional Architecture elements

This section summarises the most important non functional aspects of the system.

System Availability

The system targets **99.9%** availability. Availability is supported by static content delivery, stateless serving and automated, repeatable deployments. Health checks and monitoring detect incidents early, and rollbacks are performed by redeploying a previous immutable build.

Overview of application security

Security is addressed at several levels:

- **Build integrity** — dependencies are pinned and builds run in isolated CI jobs.
- **Runtime hardening** — the runtime image is minimal and runs with least privilege.
- **Transport security** — all traffic is served over HTTPS.
- **Content security** — no server-side execution, which keeps the attack surface small.